Self-Healing Network Operator for Kubernetes

Pod-to-Pod Data Plane Connectivity Through Synthetic Probing

Computer Networks Course Project

September 4, 2026

Abstract

Kubernetes is good at checking whether a container is running, but it does not continuously check whether one pod can reach another pod across the cluster network. This difference matters. A pod can be marked `Running` and `Ready`, while traffic from that pod to another node is silently dropped because of a CNI issue, a firewall rule, packet loss, or a broad `NetworkPolicy`.

This project builds a Kubernetes operator that measures pod-to-pod connectivity from multiple nodes. It deploys one lightweight probe agent on each node, asks these agents to probe each other, builds a directed reachability matrix, classifies the shape of failures, exposes the result through a Kubernetes custom resource, and records measurements for evaluation. The project is divided into five parts: platform setup, probe agent, operator/controller, analysis/classification, and evaluation.

The main contribution is not only detecting that the network is unhealthy. The system also tries to explain where the failure is likely located, such as a node egress problem, node ingress problem, isolated node, pair-specific failure, policy-related failure, or wider cluster partition.

Contents

1 Introduction

A Kubernetes cluster runs applications inside pods. These pods may be scheduled on different nodes. For a distributed application to work correctly, pods must be able to communicate over the cluster network. This is often called east-west traffic, meaning traffic between workloads inside the cluster.

Kubernetes already has health checks such as liveness probes and readiness probes. These checks answer questions like: "Is this container alive?" and "Should this pod receive service traffic?" They are useful, but they do not answer a different question: "Can pod A on node 1 reach pod B on node 2?"

That missing question is the problem this project studies. The cluster may look healthy from the Kubernetes control plane, but the data plane may be broken. The data plane is the actual traffic path used by application packets. If that path fails silently, users see timeouts before the platform reports a clear cause.

2 Problem Statement

The project addresses silent pod-to-pod connectivity failures in Kubernetes. A silent connectivity failure is a situation where pods and nodes appear healthy, but traffic between some pods fails or becomes degraded.

Examples include:

- a node cannot send traffic to other nodes;
- other nodes cannot send traffic to one node;
- one pair of nodes has a local connectivity issue;
- a node becomes isolated in both directions;
- a policy or firewall blocks traffic broadly;
- packets are partially lost, causing higher delay, jitter, or unstable throughput.

The project asks the following question:

> Can we continuously measure pod-to-pod connectivity, detect failures quickly, and classify the likely failure location using a simple and robust operator design?

3 Objectives

The project objectives are:

1. Build a reproducible multi-node Kubernetes test environment.
2. Deploy one probe agent per node without bypassing the pod network.
3. Collect a directed source-to-destination connectivity matrix.
4. Classify common failure patterns from the matrix.
5. Expose the current network health as Kubernetes status and Prometheus-style metrics.
6. Provide fault injection and evaluation scripts.
7. Measure non-functional requirements: performance, simplicity, and robustness.

The professor's additional non-functional requirements are handled as shown in Table 1.

Table 1: Non-functional requirements and project measurements

Requirement	What the project measures
Performance	Detection latency, probe RTT, round duration, throughput estimate
Simplicity	Probe count, bytes sent, small fixed components, bounded concurrency
Robustness	Loss rate, burst loss, jitter, agent health, safe handling of unknown states

4 Background

4.1 Kubernetes Health Checks

Kubernetes checks container health mainly through probes configured on pods. A liveness probe can restart a container if it stops responding. A readiness probe can remove a pod from service load balancing if it is not ready. These checks are local to the pod or node.

They do not continuously test every network path between pods. Therefore, a pod may be ready from Kubernetes' point of view but unreachable from another node.

4.2 Data Plane and Control Plane

The Kubernetes control plane stores desired state and cluster objects. The data plane is the actual traffic path used when one pod sends packets to another pod. This project focuses on data plane health.

The distinction is important. If the operator itself sends all probes from one central pod, then the system only tests the network from that one source. It cannot detect asymmetric faults, where node A can reach node B but node B cannot reach node A. To avoid this, the project uses distributed agents.

4.3 Related Tools

Existing tools solve parts of this problem, but not the full problem addressed here.

Table 2: Related tools and their limitations for this project

Tool or mechanism	What it does	Limitation for this project
Kubernetes liveness/readiness probes	Check whether a container responds	Do not test pod-to-pod paths across nodes
Node conditions	Report node-level health	Do not show which pod-to-pod paths fail
Goldpinger	Runs agents and shows a connectivity graph	Primarily an observability tool, not a Kubernetes status-driven operator with classification
Prometheus blackbox exporter	Runs synthetic probes	Usually probes from one location unless many targets are manually configured
Service mesh telemetry	Observes real application traffic	Passive; idle paths may stay untested

This project combines active probing, a Kubernetes custom resource, classification, and evaluation scripts in one small system.

5 System Overview

The system has four main runtime pieces:

1. a `NetworkHealthCheck` custom resource that defines what to measure;
2. a manager/operator that runs the control loop;
3. a DaemonSet of agents, one per node;
4. fault injection and evaluation scripts used during experiments.

The operator does not directly send data plane probes. Instead, it asks each source agent to probe a destination agent. This keeps the measured path close to the real pod network path.

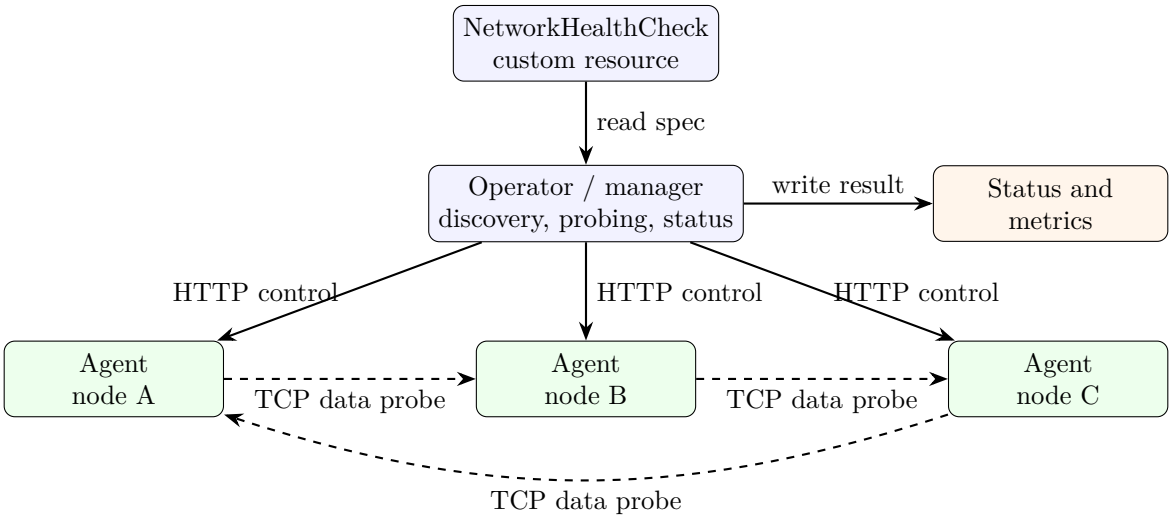

Figure 1: High-level architecture. The operator controls the experiment, while agents send the actual data-plane probes.

The result is stored as a matrix. A directed matrix means `node-a -> node-b` and `node-b -> node-a` are separate entries. This is necessary because network failures can be one-way.

	A	**B**	**C**
A	-	fail	fail
B	ok	-	ok
C	ok	ok	-

Example: row A fails.
This suggests node A
has an egress problem.

Figure 2: Directed matrix intuition. A failed row points to an outgoing traffic problem from that source.

6 P1: Platform and Deployment

P1 provides the environment in which the rest of the project runs. The repository uses KIND to create a local multi-node Kubernetes cluster. KIND is useful for this project because each

Kubernetes node is a Docker container. That makes fault injection repeatable through commands such as `iptables` and `tc netem`.

The cluster configuration is stored in `hack/kind-cluster.yaml`. The project uses one control-plane node and three worker nodes. Calico is used as the CNI plugin, and the default KIND CNI is disabled. This gives a real pod network and supports the network policy scenarios used later.

Table 3: P1 platform files

File	Purpose
`hack/setup-cluster.sh`	Creates the KIND cluster and installs Calico
`hack/reset-cluster.sh`	Tears down the local cluster
`Dockerfile`	Builds the manager image
`Dockerfile.agent`	Builds the agent image
`config/namespace.yaml`	Creates the `netprobe-system` namespace
`config/kustomization.yaml`	Applies all Kubernetes manifests together
`Makefile`	Provides common build, deploy, test, and evaluation commands

The `Makefile` keeps project operation simple. For example, `make cluster-up` creates the cluster, `make images` builds images, `make load` loads them into KIND, and `make deploy` applies the Kubernetes manifests.

7 P2: Probe Agent

P2 implements the agent that runs on every node. The agent is deployed as a DaemonSet, which means Kubernetes schedules one copy on each node. The manifest is in `config/agent/daemonset.yaml`.

Each agent exposes two ports.

Table 4: Agent ports

Port	Purpose
`8080`	Control API used by the operator
`9376`	Data-plane TCP listener used as the probe target

Keeping these ports separate is an important design choice. The control API tells the agent what to do. The data port is the actual network path being tested. If both used the same port, it would be harder to tell whether the control path failed or the data path failed.

The agent exposes the endpoints in Table 5.

Table 5: Agent endpoints

Endpoint	Meaning
`/healthz`	Reports that the agent is running and includes node/pod identity
`/probe`	Runs a probe to a requested target and returns JSON
`/metrics`	Reports a simple agent health metric

The probe endpoint accepts parameters such as target address, timeout, count, and payload size. During a probe round, the agent tries several TCP connections and writes a small payload.

It then reports success, loss rate, RTT, jitter, burst loss, bytes sent, throughput estimate, and probe counts.

This means the agent supports both functional measurement and non-functional measurement. Functional measurement answers "can it connect?" Non-functional measurement answers "how well did it connect?"

8 P3: Operator and Kubernetes API

P3 implements the operator logic. The operator periodically reads `NetworkHealthCheck` resources, discovers running agent pods, asks agents to probe each other, stores the matrix, and writes status back to Kubernetes.

The custom resource definition is in `config/crd/networkhealthcheck-crd.yaml`. A sample object is in `config/samples/networkhealthcheck-sample.yaml`.

Listing 1: Sample NetworkHealthCheck specification

```
apiVersion: network.selfheal.io/v1alpha1
kind: NetworkHealthCheck
metadata:
  name: cluster-mesh
spec:
  interval: "15s"
  probeTimeout: "2s"
  probeType: TCP
  probeCount: 5
  probePayloadBytes: 1024
  topology: FullMesh
  sampleDegree: 3
  failureThreshold: 3
  successThreshold: 2
  maxConcurrentProbes: 64
  remediation:
    enabled: true
    dryRun: false
```

The controller performs these steps:

1. List the agent pods with label `app=netprobe-agent`.
2. Ignore pods that are not running or do not have a pod IP.
3. Build directed probe pairs between the discovered agents.
4. Run probes with bounded concurrency.
5. Build a matrix keyed by `sourceNode->destinationNode`.
6. Pass the matrix to the analysis package.
7. Write the classification and reachability entries into resource status.
8. Store the latest values for Prometheus-style metrics.
9. Optionally create a remediation plan, depending on configuration.

The status fields are shown in Table 6.

The controller preserves `lastTransitionTime` when the condition has not actually changed. This is important for P5 because detection latency depends on the time when the system first moved from healthy to unhealthy.

Table 6: Important status fields

Status field	Meaning
`observedGeneration`	Version of the spec that was observed
`lastProbeTime`	Time when the latest probe round was stored
`classification`	Current classifier result
`classificationConfidence`	Confidence score from the classifier
`suspectNodes`	Nodes related to the suspected fault
`reachability`	Directed probe matrix with metrics
`conditions`	Kubernetes-style health condition

9 P4: Analysis and Classification

P4 is the analysis layer. It takes the directed matrix and returns a verdict.

The classifier is implemented in `pkg/analysis/classifier.go`. It is designed to be simple enough to explain and test. It does not need machine learning. It uses the shape of failures in the matrix.

For example, imagine three nodes: A, B, and C. If A cannot reach B or C, but B and C can still reach A, then A likely has an egress problem. In matrix form, the row for A fails. If B and C cannot reach A, but A can reach them, then A likely has an ingress problem. In matrix form, the column for A fails.

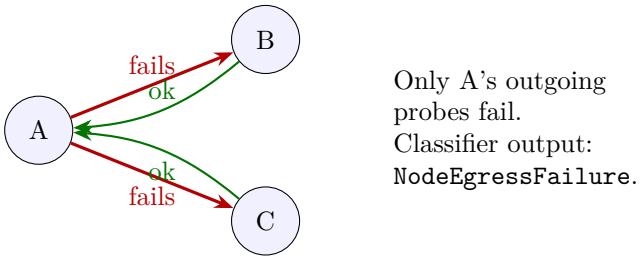

Figure 3: Classifier example for a node egress failure.

Table 7: Classifier rules

Matrix pattern	Classification
No failed pairs	`Healthy`
One directed pair fails	`PairLocalFailure`
All outgoing probes from one node fail	`NodeEgressFailure`
All incoming probes to one node fail	`NodeIngressFailure`
All incoming and outgoing probes for one node fail	`NodeIsolated`
Most or all pairs fail across the cluster	`ClusterPartition`
All pairs fail with connection refused	`PolicyScopedFailure`
Agent/control error or unclear pattern	`Unknown`

The `Unknown` class is important. It prevents the system from making a confident claim when the evidence is not clear. This is especially important if remediation is enabled.

The analysis package also contains a hysteresis tracker. Hysteresis means that the system can wait for repeated failures before marking a pair degraded, and can wait for repeated successes before marking it recovered. This reduces noise from short transient events. In the current codebase, this tracker exists as a tested analysis component, while the live controller path calls the stateless `Evaluate` entry point directly.

10 Remediation Safety

The project includes a remediation planning path in `internal/controller/remediation.go`. Remediation is intentionally conservative.

The planner refuses to act when:

- the verdict is `Healthy`;
- the verdict is `Unknown`;
- the confidence score is below `0.8`;
- the action would be unsafe for broad failures such as cluster partition or policy-scoped failure.

For node-related failures, the implemented action can restart the matching agent pod. This is a cautious remediation stub rather than a full network repair system. It shows how remediation can be wired while preserving safety guardrails.

11 Monitoring and Observability

The manager exposes `/metrics`. The current implementation writes Prometheus-style text metrics based on the latest probe round. Important metrics are listed in Table 8.

Table 8: Prometheus-style metrics

Metric	Meaning
`netprobe_manager_up`	Manager process is running
`netprobe_pair_reachable`	Whether a directed pair was reachable
`netprobe_pair_rtt_seconds`	Average pair latency
`netprobe_pair_jitter_seconds`	Pair jitter
`netprobe_pair_loss_rate`	Pair loss rate
`netprobe_pair_loss_burst`	Maximum consecutive failed probes
`netprobe_pair_throughput_bytes_per_second`	Synthetic throughput estimate
`netprobe_pair_bytes_sent`	Bytes sent in latest pair round
`netprobe_pair_probes_sent`	Probe attempts in latest pair round
`netprobe_round_duration_seconds`	Wall-clock duration of the latest round
`netprobe_mesh_health`	Whether the latest verdict is healthy
`netprobe_classification`	Latest classification confidence

The project also includes monitoring manifests under `config/monitoring/`, including a ServiceMonitor and Grafana dashboard configuration.

12 Fault Injection

To test detection and classification, the project uses controlled fault scenarios. These are implemented in `hack/inject-fault.sh`.

Table 9: Fault scenarios

Scenario	Fault	Expected classification
F1	Healthy baseline	`Healthy`
F2	Block inbound traffic on the target agent data port	`NodeIngressFailure`
F3	Block outbound traffic from the target agent data port	`NodeEgressFailure`
F4	Kill Calico node pod and isolate the agent data port	`NodeIsolated`
F5	Add 40 percent packet loss with `tc netem`	`Degraded` or degraded evidence in metrics
F6	Apply deny-all `NetworkPolicy`	`ClusterPartition` or `PolicyScopedFailure`, depending on observed error pattern

These scenarios allow the project to test both complete failures and degraded behavior. Complete failures mainly affect classification. Partial packet loss is especially useful for robustness metrics such as loss rate, jitter, and burst loss.

13 P5: Evaluation

P5 evaluates the system using repeatable experiments. The key script is `hack/measure-mttd.sh`. It clears previous faults, injects a chosen scenario, records the injection timestamp, polls the `NetworkHealthCheck` status, and writes a CSV row when an unhealthy transition is observed.

The full sweep is run with:

```
TRIALS=10 ./hack/run-p5-evaluation.sh
```

A shorter run can be used during demonstrations:

```
TRIALS=1 SCENARIOS="F2 F3 F5" ./hack/run-p5-evaluation.sh
```

The raw output is stored in `docs/results/mttd-results.csv`. The summary script `hack/summarize-results.py` produces `docs/results/p5-summary.md`.

13.1 Evaluation Metrics

13.2 Performance

Performance is measured in two ways. First, detection latency measures how long the system takes to move from injected fault to unhealthy status. Second, probe-level performance measures RTT, round duration, and throughput estimate.

This separation is useful because a system may have fast probes but slow detection if the interval is large, or it may have quick detection but expensive probe rounds if too many pairs are measured.

Table 10: CSV columns produced by the P5 scripts

Column	Meaning
`scenario`	Fault scenario ID
`trial`	Trial number
`inject_ts`	Fault injection time
`detect_ts`	Time when unhealthy status was detected
`mttd_ms`	Detection latency in milliseconds
`predicted_class`	Classifier output
`true_class`	Expected class for the injected fault
`confidence`	Classifier confidence
`avg_rtt_ms`	Average RTT across matrix entries
`avg_jitter_ms`	Average jitter across matrix entries
`avg_loss_rate`	Average loss rate across matrix entries
`max_burst_loss`	Worst burst loss observed
`avg_throughput_bps`	Average synthetic throughput
`total_bytes_sent`	Bytes sent in the round
`probe_count`	Total attempted probes

13.3 Simplicity

Simplicity is measured through design and operational cost. The system has one manager deployment and one agent per node. The probe protocol is HTTP plus TCP. The matrix format is a simple map from `source->destination` to a result object.

The measured simplicity indicators are probe count, bytes per round, and the fixed number of deployed components. These are easy to explain and easy to compare across cluster sizes.

13.4 Robustness

Robustness means the system should keep giving useful information even when the network is unstable. The project measures this using loss rate, burst loss, jitter, and control-channel errors.

The classifier treats agent control errors as `Unknown` instead of pretending they are data plane failures. This is a robustness feature because it avoids false certainty when the measurement system itself may be impaired.

14 Testing

The repository includes tests for major pure-code components:

- controller agent discovery;
- probe round matrix construction;
- bounded concurrency;
- status updates and conflict retry;
- preserving transition time for stable MTTD;
- classifier behavior over fixture matrices;
- ambiguous cases that should return `Unknown`;

- agent/control errors;
- hysteresis state transitions;
- remediation planning safety.

The fixture files in `contracts/fixtures/` describe expected matrix shapes for healthy, degraded, isolated, ingress, egress, pair-local, ambiguous, and policy-scoped cases.

15 Results Format

This report is designed to be completed with measured results after running the evaluation on the cluster. The expected generated summary table has the form shown in Table 11. It should be replaced or supplemented by `docs/results/p5-summary.md` after the experiments are run.

Table 11: Results table template

Scenario	Trials	Accuracy	Avg MTTD ms	Avg RTT ms	Avg loss rate
F2	to be measured	to be measured	to be measured	to be measured	to be measured
F3	to be measured	to be measured	to be measured	to be measured	to be measured
F4	to be measured	to be measured	to be measured	to be measured	to be measured
F5	to be measured	to be measured	to be measured	to be measured	to be measured
F6	to be measured	to be measured	to be measured	to be measured	to be measured

16 Limitations

The project has several important limitations.

First, KIND runs nodes as containers on a single host. This is excellent for reproducible testing, but it is not the same as measuring real physical machines connected by a real network. The latency and throughput numbers should therefore be treated as local experimental measurements, not as production network benchmarks.

Second, the throughput value is synthetic. The agent writes a small payload over TCP and computes bytes per second for that probe round. This is useful for comparing scenarios inside the same test setup, but it is not a replacement for a dedicated bandwidth tool such as iperf.

Third, the live controller currently builds full directed probe pairs between discovered agents. The CRD contains topology fields such as `FullMesh`, `Sampled`, and `sampleDegree`, but sampled topology is not fully implemented in the live probe planner.

Fourth, the hysteresis tracker exists in the analysis package, but the current live controller uses the stateless classifier entry point directly. This means repeated-round debouncing is available as code but not fully integrated into the live reconciliation path.

Fifth, remediation is intentionally limited. It can plan and perform conservative agent pod restarts for certain node-related cases, but it does not repair arbitrary CNI, policy, or underlay failures.

17 Future Work

The most useful next improvements are:

1. Integrate hysteresis directly into the live controller loop.
2. Fully implement sampled topology so large clusters do not require all ordered pairs every round.
3. Add real Prometheus client library registration instead of manually writing metric text.

4. Add stronger event handling so Kubernetes Events are emitted only on meaningful transitions.
5. Add a stronger remediation budget with cooldowns and per-node rate limits.
6. Add DNS probing because many real incidents are name-resolution failures rather than raw TCP failures.
7. Run the same evaluation on a larger real cluster to compare with KIND results.

18 Conclusion

This project builds a small but complete system for active pod-to-pod network health checking in Kubernetes. The main idea is simple: place an agent on each node, ask agents to probe each other, build a directed matrix, classify the matrix, and publish the result as Kubernetes status and metrics.

The design improves on basic health checks because it measures the actual paths between pods. It also improves on simple dashboards because the result becomes part of Kubernetes state and can be evaluated through repeatable experiments.

From P1 to P5, the project covers platform setup, agent implementation, operator control logic, classification, safety-aware remediation planning, and evaluation. The final system can detect connectivity failures, report where they are likely located, and measure performance and robustness indicators such as latency, throughput, loss, burst loss, and jitter.